

C  puto Concurrente 2026

Pr  ctica 8

Implementaciones de Contadores: Balance entre Correcci  n y Eficiencia

Profa: Gilde Valeria Rodr  guez

1 Contexto

Razonar sobre la correcci  n de un algoritmo concurrente es un desaf  o. Existen nociones de correcci  n que definen lo que es un comportamiento v  lido a partir de transformar una historia concurrente (con hilos ejecut  ndose al mismo tiempo) en una historia secuencial (como si ocurrieran uno tras otro) para analizar si el resultado tiene sentido.

Esta transformaci  n sigue reglas que van de m  s a menos estrictas:

- **Linealizabilidad (Linearizability [HW90]):** Es el modelo m  s fuerte. Cada operaci  n parece tener efecto de forma instant  nea y at  mica en un punto   nico en el *tiempo real*, ubicado estrictamente entre su invocaci  n y su respuesta.
- **Consistencia Secuencial (Sequential Consistency [Lam79]):** Relaja el tiempo real. Exige que exista un orden global   nico de todas las operaciones que respete el *orden de programa* de cada hilo de forma individual.
- **Consistencia en la Inactividad (Quiescent Consistency [HS08]):** No impone un orden estricto entre operaciones concurrentes superpuestas. Solo garantiza el orden a trav  s de los periodos de inactividad: si la operaci  n A termina antes de que la operaci  n B comience (separadas por un instante donde nadie hace nada), A preceder   a B.
- **Consistencia Eventual (Eventual Consistency [Vog09]):** El modelo m  s relajado. Si cesan las actualizaciones, todas las r  plicas del sistema converger  n *eventualmente* al mismo estado. Tolerar que diferentes hilos vean estados distintos temporalmente.

Los algoritmos concurrentes que garantizan linealizabilidad suelen imponer un **cuello de botella   nico** (como un candado global o una variable fuertemente sincronizada). Esto destruye la escalabilidad a medida que aumenta el n  mero de hilos.

La soluci  n propuesta en la literatura [Sha11] para alcanzar el m  ximo rendimiento es **relajar nuestras exigencias de correcci  n**. Al pasar de estructuras linealizables a estructuras con consistencia quiescente o eventual, permitimos que las operaciones se "dispersen" en el espacio de memoria, evitando que los hilos choquen entre s   y maximizando la eficiencia.

En los sistemas distribuidos masivos, la consistencia fuerte no solo es ineficiente, sino matem  ticamente imposible frente a particiones de red (Teorema CAP [GL02]). Por ejemplo, en contadores escalables a nivel global, los mensajes se atrasan, se pierden o se reordenan. Exigir linealizabilidad requerir  a detener internet por cada incremento. Por ello, se adopta la **Consistencia Eventual**: un cliente puede incrementar su contador local inmediatamente, y el protocolo se encarga de reconciliar estos valores en segundo plano para que, eventualmente, el total sea preciso [AB19].

2 Distintos Sabores de Contadores

A continuaci  n, analizaremos tres implementaciones de contadores, desde la m  s estricta y lenta, hasta las m  s relajadas y r  pidas.

2.1 Contador Linealizable

El enfoque clásico y seguro. Al delegar la operación al hardware subyacente mediante `incrementAndGet()`, garantizamos el nivel de corrección más alto.

```
1 import java.util.concurrent.atomic.AtomicLong;
2
3 public class LinearizableCounter {
4     private final AtomicLong count = new AtomicLong(0);
5
6     public void increment() {
7         count.incrementAndGet();
8     }
9
10    public long fetch() {
11        return count.get();
12    }
13 }
```

Listing 1: LinearizableCounter.

¿Por qué es Linealizable?

Porque la instrucción `incrementAndGet()` se ejecuta a nivel de procesador como una operación indivisible. Si el Hilo A y el Hilo B llaman al método al mismo tiempo, el hardware formará una fila instantánea en tiempo real.

Ejemplo de ejecución: Si A llama al método a las 10:00:01 y B a las 10:00:02, es matemáticamente imposible que B no vea el incremento de A. Al cumplir con el tiempo real, cumple automáticamente con todas las consistencias inferiores (Secuencial, Quiescente, etc.). El costo de esto es la alta contención: si 1,000 hilos llaman a `incrementAndGet()`, 999 tendrán que esperar su turno.

2.2 Sloppy Counter (Consistencia Eventual)

Este contador asigna memoria privada a cada hilo (`ThreadLocal`) y solo vuelca los datos a la variable global cuando supera un umbral. Es ultrarrápido, pero sacrifica la frescura de los datos.

```
1 public class SloppyCounter {
2     private final AtomicLong globalCount = new AtomicLong(0);
3     private final int threshold = 100;
4     private final ThreadLocal<Long> localCount = ThreadLocal.withInitial(() -> 0L);
5
6     public void increment() {
7         long current = localCount.get() + 1;
8         if (current >= threshold) {
9             globalCount.addAndGet(current);
10            localCount.set(0L);
11        } else {
12            localCount.set(current);
13        }
14    }
15
16    public long fetch() {
17        return globalCount.get(); // Lectura aproximada (eventual)
18    }
19 }
```

Listing 2: SloppyCounter.

¿Por qué es Consistente Eventualmente?

Porque si todos los hilos dejaran de incrementar y forzaran un volcado de sus variables locales, todos verían el mismo valor final correcto.

¿Por qué NO alcanza la Consistencia Secuencial?

Para ser secuencialmente consistente, el sistema global debe respetar el *orden de programa* de cada hilo.

Ejemplo de ejecución que falla:

1. El Hilo A realiza 50 incrementos. (Su código local avanza).

2. El Hilo A cambia una variable compartida: `bandera = true`.
3. El Hilo B espera a ver la bandera. Ve `bandera == true` y procede a llamar a `fetch()`.
4. El Hilo B lee 0 (porque A no alcanzó el umbral de 100 para volcar).

En una historia secuencial válida, si el Hilo B actuó *después* de ver la bandera del Hilo A, obligatoriamente debió ver los incrementos que A hizo *antes* de levantar la bandera. Como lee 0, se rompe el orden lógico.

2.3 Contador con Balanceador (Consistencia Quiescente)

El balanceador. Un *balanceador* (*balancer*) recibe tokens (hilos) y los manda alternadamente a una salida u otra: el 1.º va a la salida 0, el 2.º a la 1, el 3.º a la 0, etc. Se implementa con un `toggle` y una operación atómica de “leer y avanzar”. En quiescencia siempre se cumple $y_0 = y_1$ o $y_0 = y_1 + 1$ (*step property*), sin importar cuántos hilos pasaron ni en qué orden.

De balanceadores a contador. Si cada salida termina en un contador local $i, i + w, i + 2w, \dots$, la propiedad anterior hace que en quiescencia los valores repartidos sean exactamente $\{0, \dots, n - 1\}$. Con 3 balanceadores (raíz R , hijos L y D) y 4 hojas, seis tokens que pasan uno tras otro:

Token	Raíz	Segundo nivel	Hoja	Valor devuelto
1	$0 \rightarrow \text{izq}$	$L: 0 \rightarrow \text{izq}$	H_0	0
2	$1 \rightarrow \text{der}$	$D: 0 \rightarrow \text{izq}$	H_2	1
3	$2 \rightarrow \text{izq}$	$L: 1 \rightarrow \text{der}$	H_1	2
4	$3 \rightarrow \text{der}$	$D: 1 \rightarrow \text{der}$	H_3	3
5	$4 \rightarrow \text{izq}$	$L: 2 \rightarrow \text{izq}$	H_0	4
6	$5 \rightarrow \text{der}$	$D: 2 \rightarrow \text{izq}$	H_2	5

Las hojas se numeran de izquierda a derecha (H_0, H_1, H_2, H_3), pero reciben tokens en el orden H_0, H_2, H_1, H_3 . Por eso cada hoja arranca en un valor distinto: $H_0: 0, H_2: 1, H_1: 2, H_3: 3$, y todas suman 4.

Versión simplificada (la que usaremos). Un solo balanceador de ancho 4:

```

1 import java.util.concurrent.atomic.AtomicInteger;
2 import java.util.concurrent.atomic.AtomicLong;
3
4 public class FourWireCounter {
5     private final AtomicInteger toggle = new AtomicInteger(0); // balanceador
6     private final AtomicLong[] wire = new AtomicLong[4];
7
8     public FourWireCounter() {
9         for (int i = 0; i < 4; i++) wire[i] = new AtomicLong(i);
10    }
11
12    public long getAndIncrement() {
13        int w = toggle.getAndIncrement() & 3; // paso 1: elegir hoja
14        return wire[w].getAndAdd(4);           // paso 2: tomar valor de la hoja
15    }
16 }
```

La hoja i reparte $i, i + 4, i + 8, \dots$. Con 6 operaciones secuenciales se devuelve 0, 1, 2, 3, 4, 5. Cada operación toca solo dos variables y no hay candados.

¿Por qué es Consistente en la Inactividad? El `toggle` asigna a cada operación un número distinto, así que en quiescencia con n operaciones cada hoja repartió su parte y los valores devueltos son exactamente $\{0, \dots, n - 1\}$.

¿Por qué NO es Linealizable? Ejecución con `toggle` en 0:

1. A obtiene `toggle = 0` (hoja 0) y se **pausa** antes de tocar `wire[0]`.
2. B obtiene hoja 1 y devuelve 1. Termina.

3. *C* obtiene hoja 2 y devuelve 2. Termina.
4. *D* obtiene hoja 3 y devuelve 3. Termina.
5. *E* empieza después de que *B* terminó, obtiene hoja 0 (**wire**[0] sigue en 0) y devuelve **0**. Termina.
6. *A* se reanuda y devuelve 4.

B terminó antes de que *E* empezara, así que cualquier linealización pone a *B* antes que *E*, y en un contador secuencial el primero recibe el valor menor. Pero *B* devolvió 1 y *E* devolvió 0. Contradicción.

Sí es QC: *A* está pendiente todo el tiempo, así que nunca hay inactividad entre *B* y *E* y pueden reordenarse. La historia secuencial $E \rightarrow 0, B \rightarrow 1, C \rightarrow 2, D \rightarrow 3, A \rightarrow 4$ es válida, y al final los valores son $\{0, \dots, 4\}$.

¿Por qué NO es Secuencialmente Consistente? Con **toggle** en 0, el hilo *Q* hace una operación y devuelve 0. Después *A* obtiene hoja 1 y se pausa. *Q* hace dos más y devuelve 2 y 3. Ahora el hilo *P* hace dos operaciones seguidas: la primera cae en la hoja 0 y devuelve **4**; la segunda cae en la hoja 1, que sigue en 1 porque *A* no ha avanzado, y devuelve **1**. Al final *A* devuelve 5. En el orden de programa de *P* su primera operación precede a la segunda, pero en un contador secuencial 4 se entrega después de 1. Ningún orden global respeta esto.

3 Ejercicios

Instrucciones:

- Entrega en un **PDF** las respuestas de los ejercicios teóricos. Para los que requieran código, añade una breve descripción de los programas entregados.
- Solo un integrante del equipo debe subir la práctica. Los demás deben marcar la tarea como entregada y escribir un comentario privado con el nombre de quien entregó.
- El formato y el medio de entrega los indicará tu ayudante de laboratorio.
- Utilizar **Java 21 LTS**. Se penalizará si no compila en esta versión.

Tiempo estimado: ≈ 1.5 hr

Puntaje total: 100

1. Ejercicio 1: El Sloppy Counter Modificado

Se sugiere hacer una pequeña modificación al **SloppyCounter** para "mejorar" su consistencia. Proponer que siempre que un hilo llame al método **fetch()**, el código primero **obligue a ese mismo hilo** a volcar su **localCount** actual al **globalCount**, y recién entonces retorne el valor global.

```

1  public long fetch() {
2      long currentLocal = localCount.get();
3      if (currentLocal > 0) {
4          globalCount.addAndGet(currentLocal);
5          localCount.set(0L);
6      }
7      return globalCount.get();
8  }
9

```

Listing 3: Modificación a **fetch()**

Analiza este escenario con múltiples hilos trabajando. ¿Esta modificación hace que la estructura sea Linealizable, Secuencialmente Consistente o se mantiene Eventualmente Consistente? Argumenta tu respuesta detallando qué sucede con el estado local de los *otros* hilos que no llamaron a **fetch()**.

2. Ejercicio 2: El Contador de Dos Hojas

Considera una versión reducida del contador con balanceador, con solo dos hojas. La hoja 0 reparte 0, 2, 4, ... y la hoja 1 reparte 1, 3, 5, ...

```
1 public class TwoWireCounter {
2     private final AtomicInteger toggle = new AtomicInteger(0);
3     private final AtomicLong[] wire = { new AtomicLong(0), new AtomicLong(1) };
4
5     public long getAndIncrement() {
6         int w = toggle.getAndIncrement() & 1; // paso 1: elegir hoja
7         return wire[w].getAndAdd(2);           // paso 2: tomar valor de la hoja
8     }
9 }
```

Listing 4: TwoWireCounter.

- (a) **No es Linealizable.** Con tres hilos A , B y C , describe una ejecución en la que B termina antes de que C empiece, pero B devuelve un valor mayor que C . Indica en cada paso el valor de `toggle`, la hoja elegida y el valor que devuelve cada hilo. Explica por qué ninguna historia secuencial que respete el tiempo real puede producir esos valores.
- (b) **Sí es Consistente en la Inactividad.** Para la ejecución del inciso (a):
- Explica por qué no hay un periodo de inactividad entre B y C .
 - Da una historia secuencial válida de un contador que produzca los mismos valores.
 - Argumenta en general por qué, cuando todas las operaciones terminan, los valores devueltos son exactamente $\{0, \dots, n-1\}$.
- (c) **Cambiando el balanceador (análisis).** Se propone ahorrar la operación atómica del balanceador y escribirlo como una variable `volatile int toggle`, con `int w = toggle++ & 1;`. Antes de programar nada, analiza:
- ¿Qué pasos componen `toggle++`? Con dos hilos que ejecutan una operación cada uno, ¿puedes construir una ejecución en la que los valores devueltos al terminar *no* sean $\{0, 1\}$?
 - Según tu análisis, ¿se mantiene la unicidad de los valores devueltos? ¿Se mantiene la Consistencia en la Inactividad?
- (d) **Cambiando el balanceador (experimento).** Implementa la variante del inciso (c) y pruébala:
- Lanza varios hilos que ejecuten muchas veces `getAndIncrement()` y guarda todos los valores devueltos.
 - Cuando todos los hilos terminen (`join`), hay inactividad. Verifica si los valores son exactamente $\{0, \dots, n-1\}$: cuenta cuántos valores se repiten y cuáles faltan.
 - Repite el experimento varias veces (con distinto número de hilos y de operaciones) y reporta los resultados en una tabla.
 - Haz lo mismo con la versión original (con `AtomicInteger`) y compara.

Responde: de acuerdo con tu análisis y con tus resultados, ¿la variante sigue siendo consistente en la inactividad? Si en alguna corrida no observaste ningún error, ¿eso demuestra que la variante es correcta? Justifica.

References

- [AB19] Paulo Sérgio Almeida and Carlos Baquero. Scalable eventually consistent counters over unreliable networks. *Distrib. Comput.*, 32(1):69–89, February 2019.
- [GL02] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News*, 33(2):51–59, June 2002.
- [HS08] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2008.

- [HW90] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, July 1990.
- [Lam79] Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.
- [Sha11] Nir Shavit. Data structures in the multicore age. *Commun. ACM*, 54(3):76–84, mar 2011.
- [Vog09] Werner Vogels. Eventually consistent. *Commun. ACM*, 52(1):40–44, January 2009.